

Guia rapida para exponer: como funciona src/app/ajustes/page.jsx

Proyecto: Despacho Digital (Next.js App Router)

Archivo principal explicado: src/app/ajustes/page.jsx

Archivos relacionados:

- src/lib/preferences.js
- src/app/preferences-sync.js
- src/app/layout.js
- src/app/globals.css

1) Objetivo de la pagina Ajustes

La pagina Ajustes es un panel de configuracion del usuario.

Su funcion principal es:

- Cambiar idioma de interfaz (es, en, pt)
- Cambiar tema visual (claro, oscuro, azul)
- Guardar esas preferencias en localStorage
- Reaplicar preferencias al volver a entrar
- Mostrar y copiar datos de demo para stream (RTMP y stream key)
- Gestionar un enlace de promocion
- Simular cambio de plan de membresia

No llama APIs reales para guardar en servidor: todo se guarda en navegador.

2) Estructura general del archivo

El componente es client-side ("use client"), por eso usa:

- useState
- useEffect
- useMemo
- localStorage
- eventos de window

Importante:

- El texto visible se traduce con un objeto diccionario llamado TEXTS.
- Las preferencias visuales se aplican con funciones reutilizables del archivo src/lib/preferences.js.

3) Constantes de negocio y de demo

Constantes principales:

- RTMP_SERVER
- DEFAULT_STREAM_KEY
- DEFAULT_PROMO_LINK

Estas constantes NO levantan un servidor ni autentican OBS por si solas. Solo se usan para mostrar datos en la UI y copiarlos al portapapeles.

Tambien hay:

- PLANES: arreglo con 3 planes (basico, pro, empresa)
- DEFAULT_SETTINGS: valores iniciales del estado de ajustes

4) Diccionario de idioma (lineas aprox 50-107)

TEXTS tiene tres bloques:

- es
- en
- pt

Cada bloque contiene las mismas claves de texto, por ejemplo:

- accountSettings
- panelSettings
- platformLanguage
- themeColor
- savePreferences
- light, dark, blue

Funcion `getTexts(language)`:

- Devuelve `TEXTS[language]` si existe
- Si no existe, usa `TEXTS.es` como fallback

Como ocurre el cambio de idioma:

1. El estado idioma vive en `useState`
2. Se calcula `t = getTexts(idioma)`
3. En el JSX se renderiza `t.algo` (ejemplo: `t.panelSettings`)
4. Cuando cambia idioma, React re-renderiza y toma otro bloque de `TEXTS`

=====

5) Estados del componente

=====

Estados principales:

- idioma
- tema
- notificacionesEmail
- streamKey
- mostrarClave
- promoLink
- planActual

Estados de mensajes de feedback:

- estadoPreferencias
- estadoStream
- estadoPromo
- estadoPlan

Idea para explicarlo en clase:

"Hay estados de datos y estados de mensajes; los primeros guardan valor, los segundos confirman acciones al usuario."

=====

6) Carga inicial de ajustes (useEffect)

=====

Al montar la pagina:

1. `getStoredSettings()` lee `localStorage` con `SETTINGS_STORAGE_KEY`
2. Si hay datos, normaliza idioma y tema (seguridad de valores)
3. Rellena estados con lo guardado
4. Llama `applyVisualPreferences` para aplicar idioma/tema al documento

Resultado:

- El usuario abre Ajustes y ve lo ultimo que habia elegido.

=====

7) Guardado de ajustes

=====

Funcion clave: `guardarAjustes(overrides)`

- Arma un payload con los estados actuales
- Permite sobrescribir campos puntuales con overrides
- Guarda JSON en `localStorage`
- Lanza `notifyPreferencesUpdated()`

Ese evento permite que otras partes de la app se enteren del cambio visual.

8) Handlers importantes (interaccion)

handleIdiomaChange(event)

- Normaliza nuevo idioma
- setIdioma
- applyVisualPreferences({ idioma, tema })
- guarda en localStorage
- muestra mensaje "Idioma aplicado"

handleTemaChange(event)

- Igual idea, pero para tema
- Actualiza data-theme del html y colorScheme

handleGuardarPreferencias()

- Guarda todo y muestra "Preferencias guardadas"

handleCopiarServidor() / handleCopiarClave()

- Copian valores al clipboard
- Muestran mensaje de exito o error

handleGenerarEnlace()

- Genera codigo aleatorio con Math.random + base36
- Construye nuevo link promocional
- Lo guarda y muestra mensaje

handleCopiarEnlacePromo()

- Copia el link promocional al portapapeles

handleSeleccionarPlan(planId)

- Cambia plan actual
- Guarda en localStorage
- Muestra mensaje con nombre del plan

9) Render UI por bloques

La pagina se compone de tarjetas:

1. Preferencias generales

- Select idioma
- Select tema
- Checkbox email
- Boton guardar

2. Conexion Stream (OBS)

- Input solo lectura para servidor RTMP
- Input para stream key (mostrar/ocultar)
- Botones copiar

3. Promocion y referidos

- Input con enlace
- Botones generar y copiar

4. Estadisticas (mock)

- Numeros fijos de ejemplo

5. Membresia y precios

- Tarjetas por plan
- Resalta plan actual

10) Como se aplica realmente tema e idioma global

Archivo: src/lib/preferences.js

- normalizeLanguage y normalizeTheme validan valores permitidos
- applyVisualPreferences() cambia:
 - document.documentElement.lang
 - atributo data-theme en <html>
 - style.colorScheme

Archivo: src/app/globals.css

- Usa selectores como html[data-theme="oscuro"] para recolorear clases

Archivo: src/app/preferences-sync.js

- Escucha cambios de storage y evento custom demo:settings-updated
- Reaplica preferencias visuales para sincronizar entre vistas/pestanas

Archivo: src/app/layout.js

- Monta <PreferencesSync /> una sola vez para toda la app

11) Que SI hace y que NO hace

SI hace:

- Persistencia local de ajustes en navegador
- Cambio visual inmediato sin recargar
- Traducción local por diccionario

NO hace:

- Guardar ajustes en base de datos
- Validar stream key contra servidor real
- Emitir o recibir video real de OBS por si sola

12) Resumen para decir en 30 segundos en clase

"La pagina de Ajustes usa React state para idioma, tema y otras opciones. Cuando el usuario cambia algo, se actualiza el estado, se guarda en localStorage y se aplica en tiempo real al html con data-theme y lang. Los textos cambian porque hay un diccionario TEXTS por idioma. Ademias, hay bloques de stream/promocion/planes como simulacion funcional de pane l. Todo es local, rapido y sin backend."

13) Mini guion oral (1-2 minutos)

1. Inicio: "Es una pagina cliente de Next.js para personalizar experiencia."
2. Mostrar TEXTS: "Aqui estan los textos para es/en/pt."
3. Mostrar handlers: "Cuando cambia idioma o tema, aplico y guardo."
4. Mostrar preferences.js: "Aqui actualizo lang y data-theme del html."
5. Mostrar localStorage: "Persisto ajustes para que se mantengan al volver."
6. Cierre: "Lo de OBS en esta version es UI demo; falta backend real para stream ing real."

Fin.